

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Repaso

Motivación: NumPy

Manipulación de Listas en Python

Ejercicios Prácticos

Conclusiones

Objetivos de la Sesión 11

- **Comprender** los conceptos básicos de NumPy: arreglos (arrays), vectorización, broadcasting.
- **Practicar** la creación y manipulación de arrays en Google Colab.
- **Aplicar** estas herramientas a ejemplos numéricos y pequeños problemas de Física/Astronomía.
- **Desarrollar** habilidades de trabajo colaborativo y análisis numérico.

Introducción y Repaso

Introducción y Repaso $\ni$ Limitaciones de las Estructuras Nativas de Python

- **Rendimiento:** Las listas de Python no están optimizadas para cálculos numéricos masivos
- **Sintaxis verbose:** Operaciones matemáticas requieren bucles explícitos
 - Ejemplo: Multiplicar cada elemento de una lista por 2 $\rightarrow$ `[x*2 for x in lista]`
- **Funcionalidad limitada:** Sin soporte nativo para álgebra lineal, estadística avanzada
- **Heterogeneidad:** Las listas pueden contener diferentes tipos de datos, lo que impide optimizaciones

Problema en Ciencia y Física

Los cálculos científicos requieren alto rendimiento y operaciones optimizadas con grandes conjuntos de datos numéricos

Motivación: NumPy

Motivación: NumPy $\ni$ ¿Qué es NumPy?

- **NumPy** = **N**umerical **P**ython.
- Ofrece:
 - Estructura central: **ndarray** (arreglo multidimensional).
 - Funciones matemáticas optimizadas para operar sobre estos arreglos.
 - Alto rendimiento (implementaciones en C bajo la interfaz de Python).
- **Base de muchas librerías** científicas (p.e. SciPy, Pandas, etc.).
- **Estándar de facto** en computación científica con Python.

Historia Breve

Creada en 2005, evolucionando de bibliotecas anteriores (Numeric, Numarray), diseñada específicamente para cálculos científicos eficientes.

Motivación: NumPy $\ni$ Ventajas frente a Listas Nativas de Python

- **Vectorización:** escribir operaciones como `arr * 2` en lugar de bucles manuales.
- **Rendimiento:** implementaciones de bajo nivel (C/Fortran) para cálculos numéricos.
- **Herramientas avanzadas:** slicing sofisticado, broadcasting, manipulación de ejes (dimensiones).
- **Compatibilidad** con otras librerías (Matplotlib, SciPy, etc.).

Aplicaciones en Física

- Simulaciones numéricas (p.e. dinámica de partículas)
- Análisis de datos experimentales
- Procesamiento de señales e imágenes astronómicas
- Resolución numérica de ecuaciones diferenciales

Motivación: NumPy $\ni$ Comparativa de Rendimiento: NumPy vs. Listas

- **Ejemplo:** Suma de todos los elementos en una secuencia de un millón de números
- Con **listas de Python:** Aproximadamente 10-100 veces más lento
- Con **NumPy:** Operaciones optimizadas a nivel de hardware

Importancia del Rendimiento

En problemas físicos reales (simulaciones, análisis de datos experimentales), la diferencia de rendimiento puede significar horas o incluso días de tiempo de cómputo.

Hoy aprenderemos: Cómo aprovechar esta potencia para nuestros problemas científicos.

Manipulación de Listas en Python

Manipulación de Listas en Python $\ni$ Creación y Acceso a Listas

- Las listas son **colecciones ordenadas y mutables**
- Creación y acceso:

```
1  # Creación de listas
2  numeros = [10, 20, 30, 40, 50]
3  nombres = ["Ana", "Carlos", "Elena"]
4  mixta = [1, "dos", 3.0, True]
5
6  # Acceso por índice (desde 0)
7  print(numeros[0])      # 10
8  print(numeros[-1])     # 50 (último elemento)
9
10 # Slicing [inicio:fin:paso]
11 print(numeros[1:4])     # [20, 30, 40]
12 print(numeros[:2])      # [10, 30, 50] (saltos de 2)
```

Manipulación de Listas en Python $\ni$ Métodos para Agregar y Quitar Elementos

```
1  frutas = ["manzana", "naranja", "plátano"]
2
3  # Agregar elementos
4  frutas.append("uva")          # Al final
5  print(frutas) # ['manzana', 'naranja', 'plátano', 'uva']
6
7  frutas.insert(1, "pera")      # En posición específica
8  print(frutas) # ['manzana', 'pera', 'naranja', 'plátano', 'uva']
9
10 # Eliminar elementos
11 eliminado = frutas.pop()      # Elimina y devuelve el último
12 print("Eliminado:", eliminado) # Eliminado: uva
13 print(frutas) # ['manzana', 'pera', 'naranja', 'plátano']
14
15 frutas.remove("pera")         # Elimina por valor
16 print(frutas) # ['manzana', 'naranja', 'plátano']
17
18 del frutas[0]                 # Elimina por índice
19 print(frutas) # ['naranja', 'plátano']
```

Manipulación de Listas en Python $\ni$ Unir y Separar Listas

```
1  # Unir listas
2  lista1 = [1, 2, 3]
3  lista2 = [4, 5, 6]
4
5  # Concatenación con +
6  combinada = lista1 + lista2
7  print(combinada) # [1, 2, 3, 4, 5, 6]
8
9  # Método extend
10 lista1.extend(lista2)
11 print(lista1)    # [1, 2, 3, 4, 5, 6]
12
13 # Multiplicar listas
14 repetida = [0] * 4
15 print(repetida)  # [0, 0, 0, 0]
16
17 # Dividir cadena en lista
18 texto = "Python es poderoso"
19 palabras = texto.split()
20 print(palabras)  # ['Python', 'es', 'poderoso']
21
22 # Unir lista en cadena
23 unida = " | ".join(palabras)
24 print(unida)     # 'Python | es | poderoso'
```

Manipulación de Listas en Python $\ni$ Métodos Adicionales para Listas

```
1  numeros = [3, 1, 4, 1, 5, 9, 2]
2
3  # Ordenar lista (modifica la original)
4  numeros.sort()
5  print(numeros)  # [1, 1, 2, 3, 4, 5, 9]
6
7  # Invertir lista
8  numeros.reverse()
9  print(numeros)  # [9, 5, 4, 3, 2, 1, 1]
10
11 # Contar ocurrencias
12 print(numeros.count(1))  # 2
13
14 # Encontrar índice (primera aparición)
15 print(numeros.index(5))  # 1
16
17 # Copiar una lista
18 lista_copia = numeros.copy()  # o list(numeros) o numeros[:]
19 print(lista_copia)  # [9, 5, 4, 3, 2, 1, 1]
20
21 # Verificar si existe un elemento
22 print(5 in numeros)  # True
23 print(7 in numeros)  # False
```

Manipulación de Listas en Python $\ni$ Listas por Comprensión

- Forma concisa de crear listas basadas en listas existentes
- Estructura: `[expresión for item in iterable if condición]`

```
1  # Lista del cuadrado de números del 0 al 9
2  cuadrados = [x**2 for x in range(10)]
3  print(cuadrados)  # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
4
5  # Lista de números pares del 0 al 9
6  pares = [x for x in range(10) if x % 2 == 0]
7  print(pares)      # [0, 2, 4, 6, 8]
8
9  # Combinar elementos de dos listas
10 nombres = ["Ana", "Luis"]
11 apellidos = ["Gómez", "Pérez"]
12 nombres_completos = [n + " " + a for n in nombres for a in apellidos]
13 print(nombres_completos)
14 # ['Ana Gómez', 'Ana Pérez', 'Luis Gómez', 'Luis Pérez']
```

Manipulación de Listas en Python $\ni$ Caso Práctico con Listas: Análisis de Datos

```
1  # Temperaturas diarias de una semana (°C)
2  temperaturas = [22.5, 23.1, 21.8, 24.2, 25.0, 22.9, 21.5]
3
4  # Análisis básico manual
5  suma = sum(temperaturas)
6  prom = suma / len(temperaturas)
7  temp_max = max(temperaturas)
8  temp_min = min(temperaturas)
9
10 print(f"Promedio: {prom:.1f}°C")
11 print(f"Máxima: {temp_max}°C, Mínima: {temp_min}°C")
12 print(f"Rango: {temp_max - temp_min:.1f}°C")
13
14 # Identificar días con temperaturas > 23°C
15 dias_calurosos = [i+1 for i, t in enumerate(temperaturas) if t >
    ↪ 23]
16 print(f"Días calurosos: {dias_calurosos}") # [4, 5]
```


Ejercicios Prácticos

Ejercicios Prácticos $\ni$ Ejercicio 1:

Primer Contacto con NumPy



Enunciado

- Crear un arreglo 1D con los números del 1 al 10.
- Imprimir su **media**, **suma** y **producto** de todos los elementos.
- Reemplazar los valores ≤ 5 por 0, y los restantes por 1 (opcional: slicing lógico).
- Mostrar el arreglo resultante.

extbfConceptos: Arrays, operaciones vectorizadas, slicing lógico.

extbfFísica relevante: Manipulación de datos experimentales.

extbfSugerencia: Usa **`np.arange`** y operaciones de comparación (**`arr <= 5`**).



Enunciado

- Crear una matriz 3x3 con $\{[1, 2, 3], [4, 5, 6], [7, 8, 9]\}$.
- Definir un **vector** $[10, 20, 30]$.
- Sumar dicho vector a cada fila de la matriz (broadcasting).
- Mostrar el resultado.

extbfConceptos: Manipulación 2D, broadcasting, suma de vectores.

extbfFísica relevante: Suma de señales o datos experimentales.

Ejercicios Prácticos $\ni$ Ejercicio 3:

Aproximación de π con Serie



Enunciado

- Usar `np.arange` para generar un rango de n valores.
- Calcular una aproximación de π usando la serie de Leibniz:

$$\pi \approx 4 \sum_{k=0}^n \frac{(-1)^k}{2k+1}$$

- Comparar el valor obtenido para distintos n .

extbfConceptos: Vectorización, sumas numéricas, aproximación de constantes.

extbfFísica relevante: Métodos numéricos en física y matemáticas.

extbfSugerencia: Emplear vectorización para computar $\frac{(-1)^k}{2k+1}$.

- Dividir la clase en **parejas o tríos**.
- Resolver los ejercicios en un **notebook de Colab**.
- Explorar ejemplos adicionales:
 - Generar **matrices aleatorias** (`np.random.rand()`) y calcular su determinante (**opcional, con `np.linalg.det`**).
 - Practicar slicing y reemplazo de submatrices.

- ¿Problemas de instalación o importación en Colab?
- ¿Dificultad para entender la forma (**shape**) de un arreglo?
- ¿Alguna confusión con slicing, vistas y copias?

Comparte tus dudas o experiencias, discutimos en conjunto.

Ejercicios Prácticos $\ni$ Solución 1 de Referencia:

Primer Contacto con NumPy



```
1  # Creamos el arreglo y calculamos estadísticas
2  import numpy as np
3  arr = np.arange(1, 11)  # [1..10]
4  print("Array:", arr)
5  print("Media:", np.mean(arr))    # 5.5
6  print("Suma:", np.sum(arr))      # 55
7  print("Producto:", np.prod(arr))# 3628800
8
9  # Reemplazo valores <=5 con 0, y >5 con 1
10 arr_mod = arr.copy()
11 arr_mod[arr_mod <= 5] = 0
12 arr_mod[arr_mod > 5] = 1
13 print("Array modificado:", arr_mod)
```

extbfDiscusión: El uso de slicing lógico permite modificar el arreglo de forma eficiente y sin bucles.

Ejercicios Prácticos $\ni$ Solución 2 de Referencia: Matriz y Broadcasting



```
1  # Suma de vector a matriz usando broadcasting
2  import numpy as np
3  mat = np.array([[1,2,3],
4                  [4,5,6],
5                  [7,8,9]])
6  fila = np.array([10, 20, 30])
7  resultado = mat + fila # Broadcasting
8  print("Matriz resultante:\n", resultado)
9  # [[11 22 33]
10 #  [14 25 36]
11 #  [17 28 39]]
```

extbfDiscusión: Broadcasting permite sumar el vector a cada fila sin bucles explícitos.

Ejercicios Prácticos $\ni$ Solución 3 de Referencia:

Aproximación de π



```
1  # Aproximación de pi usando la serie de Leibniz
2  import numpy as np
3
4  n = 1000000 # término superior
5  k = np.arange(n+1) # [0..n]
6
7  # (-1)^k = np.power(-1, k)
8  # denominador = (2*k + 1)
9  terminos = ((-1)**k) / (2*k + 1)
10 aprox_pi = 4 * np.sum(terminos)
11 print("Aproximación de pi con n =", n, ":", aprox_pi)
12
13 # Comparar con valor real
14 import math
15 print("Diferencia:", abs(math.pi - aprox_pi))
```

Discusión: La vectorización permite calcular sumas grandes de manera eficiente y rápida.

- **Verificación:** ¿Coincidió con tus resultados o surgieron discrepancias?
- **Rendimiento:** usar vectorización en lugar de bucles manuales.
- **Optimización:** manipulación y slicing evitan copias innecesarias.

Conclusiones

- **Aprendimos** los conceptos fundamentales de NumPy y su utilidad para cálculos numéricos.
- **Practicamos** la creación, manipulación y operaciones vectorizadas con arrays.
- **Exploramos** aplicaciones en problemas matemáticos y de simulación.
- **Desarrollamos** habilidades de análisis numérico y trabajo colaborativo.

Habilidad adquirida

Uso eficiente de arrays y operaciones vectorizadas para resolver problemas científicos.

- **Sesión siguiente:** Operaciones avanzadas con NumPy (reshaping, álgebra lineal) y visualización con Matplotlib.
- **Recomendación de estudio:** Repasa la documentación oficial de NumPy y experimenta con ejemplos propios.
- **Práctica sugerida:** Resuelve ejercicios adicionales con datos reales o simulados.

extbf;Nos vemos en la próxima sesión!